

SIPLUG CHECKOUT API

Documentação técnica e arquitetural

Arquitetura • Segurança • Operação • Evolução

Retrato do código-fonte em 21/07/2026

Esta documentação distingue capacidades implementadas de lacunas atuais. Itens marcados como não implementados não constituem contrato público da API.

MONÓLITO MODULAR	6 GATEWAYS	API V1
NestJS + Prisma	MP • PagSeguro • PicPay • PayPal • Stripe • InfinitePay	HTTP síncrono

Estado desta documentação: retrato do código-fonte em 21 de julho de 2026. Onde uma capacidade não está implementada, ela é indicada explicitamente como não implementada; não deve ser considerada um contrato público.

O catálogo detalhado de todos os fluxos, campos de entrada e rotas está em [DOCUMENTACAO_FLUXOS.txt](#).

1. Visão geral

O SIPLUG Checkout API é um backend de checkout multiempresa (**Office**) que centraliza a criação de sessões de checkout, clientes de pagamento, transações, assinaturas e regras de split. A aplicação persiste seu próprio registro operacional e encaminha cobranças aos provedores configurados por credenciais.

Os fluxos implementados abrangem cartão tokenizado e métodos suportados pelo gateway selecionado, pagamentos recorrentes, captura/retorno de PayPal, atualização de status, recebimento de webhooks e transferências de split (Stripe). Os adaptadores de pagamento existentes são: Mercado Pago, PagSeguro, PicPay, PayPal, Stripe e InfinitePay. A disponibilidade real de cada método depende da credencial e da capacidade configurada no gateway.

Consumidores previstos:

- backoffice SIPLUG para cadastros e consultas administrativas;
- frontend/mobile de checkout, que cria sessão e solicita pagamento;
- gateways de pagamento, que chamam os endpoints de webhook;
- processos internos que geram faturas e processam recorrências.

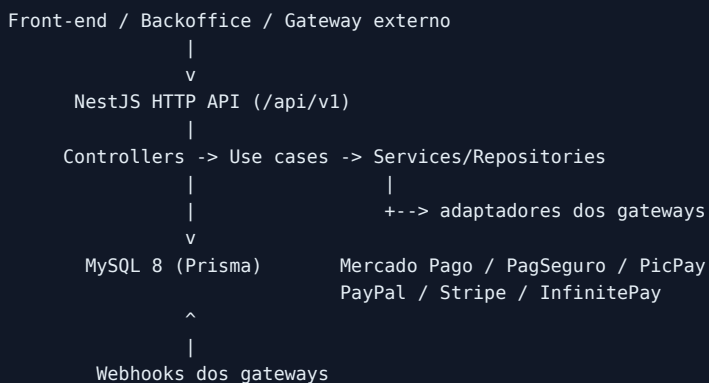
Glossário

Termo	Significado no sistema
Office	Contexto organizacional/tenant ao qual os recursos pertencem.
Client	Cliente de aplicação/integração associado a um Office .
PaymentCustomer	Comprador/pagador salvo para cobranças e assinaturas.
CheckoutSession	Intenção de compra com valor, itens, URLs de retorno e gateway.
PaymentTransaction	Registro local de uma tentativa/processamento de pagamento.
Gateway	Provedor externo de pagamento e seu adaptador de integração.
ApiCredential	Credencial de um gateway, associável a Office, Client e Gateway, em sandbox ou produção.
Split	Distribuição de parte de um pagamento entre recebedores/regras.
Assinatura	Cobrança recorrente baseada em plano e cliente de pagamento.
unique_id / _id	UUID público usado nas relações e nas chamadas da API; não é o ID numérico interno.
Token	Campo de autenticação/identificação aceito por diversos casos de uso; não há guarda global que o valide.

2. Arquitetura e design

Visão arquitetural

É um monólito modular NestJS, HTTP síncrono, com persistência em MySQL via Prisma. Não há fila, worker, cache, broker de mensagens ou API Gateway declarados no repositório.



O bootstrap habilita `rawBody`, necessário à validação de assinaturas de webhook, JSON e URL-encoded de até 10 MB e `ValidationPipe` global com transformação e remoção de campos não declarados. O prefixo padrão é `/api/v1`; a exceção é `POST /w/p/:apiCredentialId`, webhook curto do PagSeguro.

Stack

Camada	Tecnologia
Linguagem/runtime	TypeScript / Node.js
Framework HTTP	NestJS 11 (Express)
Persistência	Prisma 7 + adaptador MariaDB, banco MySQL 8
Validação	<code>class-validator</code> e <code>class-transformer</code>
HTTP para provedores	<code>@nestjs/axios</code> / <code>fetch</code> nos adaptadores
Criptografia local	AES-256-GCM para segredos de credenciais; bcrypt para senhas
Contêiner local	Docker Compose (app + MySQL 8)

Organização de código

```
src/
├─ common/           # criptografia, logs, tratamento de exceções e utilitários
├─ infra/database/prisma/ # PrismaService e módulo de banco
├─ modules/          # entidades, repositórios e serviços por domínio
│   └─ gateway-orchestration/
│   └─ payment-*/     # clientes, transações, splits, webhooks
│   └─ subscriptions/
├─ use-cases/        # controller + DTO/request + caso de uso de cada operação HTTP
├─ app.module.ts
├─ main.ts
└─ prisma/schema.prisma # modelo relacional e migrações
```

O desenho é próximo de uma arquitetura em camadas: controllers adaptam HTTP; casos de uso orquestram regras; services/repositories isolam dados; providers isolam particularidades de gateways. As interfaces de repositório e tokens de injeção reduzem o acoplamento com Prisma.

3. Segurança, autenticação e dados sensíveis

Situação atual

- Autenticação/autorização HTTP global: não implementada. Não existem guards, JWT, OAuth2 ou middleware de API key aplicados aos controllers. Alguns payloads aceitam `token` e `POST /payments/process` também lê `Authorization: Bearer <token>`, mas esta documentação não os trata como proteção suficiente: a validação ocorre apenas onde cada caso de uso a utiliza.
- Existe serviço de verificação de permissões por posição (`administrator` ou permissões por ação/entidade), mas ele não está conectado globalmente às rotas HTTP.
- Não há CORS, rate limiting, correlação de requisição (`X-Request-ID`) ou limitação de payload além de 10 MB configurados no bootstrap.

Implicação: esta API não deve ser exposta publicamente antes de implementar autenticação, autorização por tenant/Office, TLS no proxy de borda, rate limit e auditoria de acesso.

Segredos e cartão

- Credenciais de gateway são criptografadas com AES-256-GCM. `CRYPT_KEY` é obrigatória e deve conter exatamente 32 bytes; nunca a versione nem a envie ao cliente.
- Há fluxos de tokenização de cartão para gateways específicos. A integração deve enviar token/dados exigidos pelo gateway e não registrar PAN/CVV em `metadata`, `config` ou logs.
- A aplicação tenta mascarar `token`, `authorization`, `password`, `secret`, chaves e headers no contexto de logs de exceções. Isto não substitui política de retenção e controle de acesso aos logs.
- A responsabilidade PCI-DSS não é declarada no repositório. Como os métodos e payloads variam por gateway, trate a solução como não certificada/documentada para PCI até que haja avaliação formal. Prefira tokenização hospedada pelo gateway.

Ambientes

Cada `ApiCredential` possui `environment`, cujo padrão no modelo é `sandbox`. A URL pública de sandbox e a URL de produção não são definidas no repositório; devem ser fornecidas pelo deploy/proxy. Não é seguro inventá-las nesta documentação.

4. Convenções da API

- Base local: `http://localhost:3000/api/v1` (ou `APP_PORT`; no Compose, host `http://localhost:8085/api/v1`).
- Comunicação: JSON UTF-8; webhooks também podem chegar em payload bruto/URL-encoded conforme o gateway.
- `Content-Type: application/json` é necessário para requests JSON.
- UUIDs devem ser enviados nos campos terminados em `Id` quando o recurso já existe.
- Valores monetários são inteiros positivos (`amount`, `unitAmount`, `installmentAmount`); o código não documenta uma escala universal. Adote e valide a unidade monetária junto ao integrador antes de produção (usualmente centavos).

- A maioria dos **POST** usa resposta padrão NestJS 201; **POST /payments/process** declara 200. Os **PUT** usam 200, salvo comportamento customizado futuro.
- Não há Swagger/OpenAPI configurado e nem contrato de API versionado além do prefixo **/api/v1**.

Endpoint de saúde

Método	Rota	Retorno
GET	/api/v1/health	200 com { "status": "success", "data": { "app": "siplug-checkout-api", "message": "checkout api is running" } }

Catálogo de operações

Todas as rotas abaixo recebem o prefixo **/api/v1**, exceto onde indicado. Rotas **dev/*** são auxiliares de tokenização e não devem ser expostas em produção.

Domínio	Operações										
Usuários, posições e permissões	`POST /users/register`	list	get	get-all-by-office-id, PUT /users/update; POST /positions/register	get-by-unique-id	list-by-office-id	update-by-unique-id; POST /permissions/register	get-by-unique-id	list-by-office-id	update-by-unique-id; POST /position-permissions/sync	list-by-position-id`
Gateways e credenciais	`POST /gateways/register`	get-all, PUT /gateways/update; POST /api-credentials/register	get-by-unique-id	list-by-office-id, PUT /api-credentials/update`							
Clientes de pagamento	`POST /payment-customers/register`	list	get-by-unique-id	get-by-office-id; PUT /payment-customers/update`							

Domínio	Operações										
Sessões	`POST /checkout-sessions/register	list	get-by-unique-id	list-by-office-id; PUT /checkout-sessions/update`							
Pagamentos/transações	`POST /payments/process	dispatch-to-gateway	process-recurring; POST /payment-transaction/list	get-by-unique-id	list-by-office-id	sync-status; PUT /payment-transactions/update`					
Assinaturas	`POST /subscriptions/register	list	get-by-unique-id	list-by-office-id, PUT /subscriptions/update; POST /subscriptions/register	list	get-by-unique-id	list-by-office-id, PUT /subscriptions/update; POST /subscription-invoices/generate	list	get-by-unique-id	list-by-office-id, PUT /subscription-invoices/update`	
Split	`POST /split-recipients/register	list	get-by-unique-id	list-by-office-id, PUT /split-recipients/update; POST /split-rules/register	list	get-by-unique-id	list-by-office-id, PUT /split-rules/update; POST /split-rule-recipients/sync; POST /payment-split/calculate	register	get-by-unique-id	list-by-office-id	list-by-payment-transaction-id; PUT /payment-splits/update-lifecycle`

Domínio	Operações										
PayPal	GET /paypal/checkout/return/:apiCredentialId e GET /paypal/checkout/cancel/:apiCredentialId										
Desenvolvimento	POST /dev/gateway-card-tokens/mercado-pago/create; GET /dev/mercado-pago/card-token-page; GET /dev/pagseguro/encrypted-card-page/:apiCredentialId; GET /dev/picpay/temporary-card-token-page/:apiCredentialId										

Fluxo principal: criar sessão e processar pagamento

1. Cadastre/tenha um `Office`, `Client`, `Gateway` e `ApiCredential` ativos.
2. Opcionalmente crie um `PaymentCustomer`.

3. Crie `CheckoutSession` em `POST /checkout-sessions/register` com itens e valor.
4. Tokenize o cartão no gateway quando o método exigir e preserve apenas o token no cliente.
5. Chame `POST /payments/process` com a sessão, o método e os dados específicos do gateway.
6. Use `POST /payment-transactions/sync-status` ou aguarde webhook para conciliar estados assíncronos.

POST /checkout-sessions/register

Campos obrigatórios: `officeId`, `clientId`, `paymentType`, `amount` (inteiro ≥ 1) e `items`.

```
{
  "officeId": "uuid-office",
  "clientId": "uuid-client",
  "paymentCustomerId": "uuid-customer",
  "gatewayId": "uuid-gateway",
  "apiCredentialId": "uuid-credential",
  "externalReference": "pedido-1001",
  "idempotencyKey": "checkout-pedido-1001",
  "paymentType": "card",
  "amount": 15990,
  "currency": "BRL",
  "description": "Pedido 1001",
  "successUrl": "https://loja.exemplo/pagamento/sucesso",
  "cancelUrl": "https://loja.exemplo/pagamento/cancelado",
  "items": [{ "name": "Produto", "quantity": 1, "unitAmount": 15990 }]
}
```

Itens exigem `name` e `unitAmount`; `quantity` é opcional e deve ser ≥ 1 . Os campos `metadata` e `config` aceitam objeto JSON, mas não são local para dados de cartão/segredos.

POST /payments/process

O endpoint retorna 200 e entrega a sessão e a transação local. Envie `token` no corpo ou `Authorization: Bearer <token>` quando o fluxo o exigir. A aplicação não aceita `Idempotency-Key` como header próprio: nesta rota a chave é o campo JSON `idempotencyKey`.

```
{
  "checkoutSessionId": "uuid-session",
  "paymentMethod": "card",
  "installments": 1,
  "idempotencyKey": "payment-pedido-1001",
  "externalReference": "pedido-1001",
  "gatewayProvider": "mercado-pago",
  "apiCredentialId": "uuid-credential",
  "payer": { "email": "cliente@exemplo.com" },
  "paymentData": { "token": "token-fornecido-pelo-gateway" },
  "metadata": { "orderId": "1001" }
}
```

Obrigatórios: `checkoutSessionId` e `paymentMethod`. Opcionais: `installments` (≥ 1), `installmentAmount` (≥ 1), `interestAmount` (≥ 0), `interestType`, `idempotencyKey`, `externalReference`, `payer`, `paymentData`, `gatewayProvider`, `gatewaySlug`, `gatewayId`, `apiCredentialId`, `metadata` e `config`.

O formato de sucesso é o DTO `{ "checkoutSession": { ... }, "paymentTransaction": { ... } }`. Seus campos variam com gateway e método; a transação contém, entre outros, identificadores local/externo, valores, método, status e chave de idempotência. O consumidor deve persistir o `_id` e o `externalReference`, não inferir aprovação apenas pelo código HTTP.

5. Erros, resiliência e idempotência

Erros HTTP

Não há filtro global que transforme todas as exceções em um envelope de negócio uniforme. A validação do Nest normalmente retorna 400 com `message`, `error` e `statusCode`; casos de uso podem retornar/expor mensagens próprias. Portanto, clientes devem tratar HTTP e não depender de um campo `code` estável.

Status	Uso esperado
200	Consulta/atualização bem-sucedida e <code>payments/process</code> .
201	Criação via POST sem <code>@HttpCode</code> explícito.
400	Payload inválido, campo obrigatório ausente ou regra de negócio/integração rejeitada.
404	Recurso não encontrado, quando o caso de uso o emitir.
409	Não há contrato global; use apenas se o endpoint vier a declarar conflito.
500	Falha inesperada, banco, configuração ou integração não tratada.
502/504	Não há mapeamento global confirmado; um proxy/deploy pode produzi-los em falha/timeout de gateway.

Exemplo típico de erro de validação Nest (não é envelope de domínio garantido):

```
{ "statusCode": 400, "message": ["amount must not be less than 1"], "error": "Bad Request" }
```

Códigos de negócio

Não há enum nem tabela de códigos internos estáveis implementada. Mensagens como recusa de cartão, saldo insuficiente ou timeout dependem da normalização de cada gateway. Não use `CARD_DECLINED`, `INSUFFICIENT_FUNDS`, `GATEWAY_TIMEOUT` ou equivalentes como contrato até que sejam implementados e publicados.

Idempotência

O sistema persiste `idempotencyKey` em sessão/transação e encaminha a chave aos provedores quando suportado (por exemplo, cabeçalhos de Mercado Pago/Stripe/PagSeguro/PicPay/PayPal). Recorrências usam chave determinística no formato `recurring-<checkoutSessionId>-<invoiceId>`.

Limitações importantes do contrato atual:

- a chave é recebida no corpo JSON em `idempotencyKey`, não no header `Idempotency-Key` da API;
- não há evidência de uma tabela/fluxo de replay que devolva exatamente a primeira resposta da API para mesma chave;
- não existe TTL/expiração documentado no código;
- logo, a garantia de não duplicação é parcial e depende do gateway selecionado.

Para produção, implemente reserva atômica por (`officeId`, `idempotencyKey`), hash do request, resposta persistida e TTL explícito (por exemplo, 24 h), retornando 409 se a mesma chave vier com payload divergente.

6. Webhooks e notificações assíncronas

Esta aplicação recebe webhooks dos gateways; ela não contém um módulo que envie webhooks próprios aos clientes, nem política de reentrega de saída.

Gateway	Rota de recebimento
Stripe	POST /api/v1/webhooks/stripe/:apiCredentialId
Mercado Pago	POST /api/v1/webhooks/gateways/mercado-pago ou /mercado-pago/:apiCredentialId
PagSeguro	POST /api/v1/webhooks/gateways/pagseguro/:apiCredentialId ou POST /w/p/:apiCredentialId (sem prefixo)
PicPay	POST /api/v1/webhooks/gateways/picpay/:apiCredentialId
PayPal	POST /api/v1/webhooks/gateways/paypal/:apiCredentialId
InfinitePay	rotas sob POST /api/v1/webhooks/gateways/... definidas pelo controller específico
Genérico	POST /api/v1/webhooks/gateways/:gatewayProvider

As validações implementadas são específicas por provedor: Stripe usa [Stripe-Signature](#) e HMAC/timestamp com tolerância padrão de 300 s; Mercado Pago usa [x-signature](#); PagSeguro valida [x-authenticity-token](#) conforme o modo da credencial; PicPay, PayPal e demais usam seus validadores/adaptadores. O [rawBody](#) é preservado para essas verificações. Configure os segredos no ambiente/credencial correspondente e nunca aceite evento sem a validação do adaptador.

Eventos recebidos são persistidos e marcados como [processing](#), [processed](#) ou [failed](#), o que suporta rastreabilidade de processamento. Não há documentação de um payload unificado porque cada gateway envia formato próprio. Também não há retry de webhooks enviados pela API, pois envio de saída não existe.

7. Execução local e configuração

Pré-requisitos

- Node.js compatível com NestJS 11 (recomendado: Node 20+);
- npm;
- Docker e Docker Compose, se for usar o MySQL local;
- uma [CRYPT_KEY](#) com 32 bytes para operações que criam/leem credenciais de gateway.

Variáveis necessárias

Crie [.env](#) local (não há [.env.example](#) versionado). O mínimo identificável é:

```
APP_PORT=3000
APP_ENV=local
APP_NAME=siplug-checkout-api
DATABASE_URL="mysql://siplug_checkout_user:siplug_checkout_password@localhost:3335/siplug_checkout"
CRYPT_KEY="uma-chave-com-exatos-32-bytes!!"
```

O Compose inicia MySQL em [localhost:3335](#) com banco [siplug_checkout](#), usuário [siplug_checkout_user](#) e senha [siplug_checkout_password](#). Os provedores exigem segredos/configurações próprios nas [ApiCredentials](#); Stripe também requer [STRIPE_WEBHOOK_SECRET](#)

para validar seu webhook.

Comandos

```
npm install
docker compose up --build

# alternativa sem Docker para a aplicação
npm run start:dev

# validação
npm run build
npm test
npm run test:e2e
```

Antes de iniciar contra banco vazio, aplique as migrações Prisma apropriadas ao fluxo da equipe. O repositório contém migrações em [prisma/migrations](#); não há script npm de migração definido.

8. Sandbox, mocks e dados de teste

O repositório não publica URLs de sandbox/produção, cartões de teste, PIX/boleto de teste ou mocks de aprovação/recusa. Esses dados devem ser consultados na documentação oficial do gateway selecionado e cadastrados como [ApiCredential](#) com `environment: "sandbox"`.

Não use números reais de cartão no ambiente local. Para testes automatizados, simule a camada de provider e valide as transições locais; para integração, use tokens/cartões de teste do próprio provedor. As páginas e endpoints [dev/*](#) são auxiliares de tokenização, não uma garantia de sandbox universal.

9. Logs, auditoria e evolução

- O modelo inclui [changes_history](#) em diversas entidades, incluindo transações, sessões, clientes e credenciais; há um serviço para montá-lo.
- O tratamento de exceções registra contexto sanitizado e pode encaminhá-lo a um provedor de logs configurado. Há fallback de log no console.
- Não existe [X-Request-ID/Trace-ID](#) de entrada ou resposta. Para suporte atual, registre [_id](#), [externalReference](#), ID externo do gateway e timestamp.
- Não há política publicada de versionamento/depreciação. A versão de rota atual é [v1](#); qualquer breaking change deve criar [/api/v2](#), manter [v1](#) por prazo comunicado e documentar migração antes do desligamento.

Ciclo de vida

Os modelos usam campos textuais de status e [process_status](#), em vez de enum central publicado. Os gateways normalizam seus próprios estados e webhooks/sincronização os atualizam. Assim, não há uma máquina de estados única e confiável a declarar como contrato. Uma leitura segura do fluxo é:

```
CheckoutSession criada
-> PaymentTransaction criada/processada
-> gateway responde ou webhook/sync atualiza
-> estado final depende do gateway e de regras de negócio
```

Antes de integrar uma automação financeira, valide os valores efetivos de status para o gateway/método escolhido e trate estados desconhecidos como não finais.

10. Lacunas a resolver antes de publicar a API para terceiros

1. Aplicar autenticação e autorização por Office/Client em todas as rotas administrativas e de pagamento.
2. Publicar OpenAPI/Swagger com schemas e exemplos de resposta reais por endpoint.
3. Definir envelope de erro e códigos de domínio estáveis.
4. Implementar idempotência de entrada com replay/TTL e documentá-la.
5. Adicionar rate limiting, CORS/TLS de borda e [X-Request-ID](#).
6. Publicar URLs oficiais e dados de teste por ambiente.
7. Definir estados canônicos de transação, retenção de logs/auditoria e política de depreciação.